



Mulungushi University  
School of Engineering and Technology  
Department of Computer Science and IT

Practical: Inspecting a Live Web Page

**Name:** Dada mshisha

**Student ID:** 202200670

**Course code:** ICT-461

**Date:**

## 1. Objective

Following the Unit 1 lecture task, this exercise required opening a page used often, inspecting a heading, a button, and a form field, checking whether images have alternative text, opening the Network panel and reloading the page, and identifying the slowest request on that page. Two familiar sites were inspected for this task: the Mulungushi University Moodle eLearning Environment (moodle.mu.ac.zm) and YouTube.

## 2. Page 1 — Moodle eLearning Environment

The Moodle login page was opened with DevTools' Network panel active. The Username and Password form fields and the Log in button were inspected, and the page was reloaded to capture the resulting network requests.

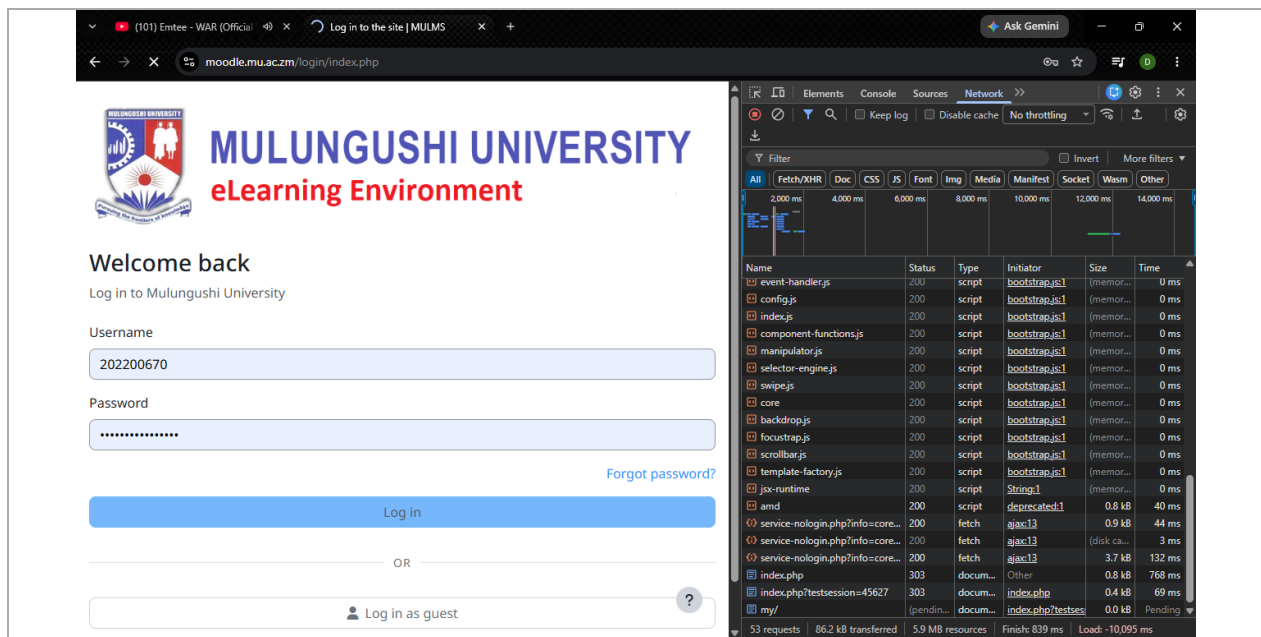


Fig. 1 — Moodle login page: Username field, Password field, and Log in button, with the Network panel recording the page reload.

After logging in, the Moodle dashboard and a course page were also inspected to compare how many additional requests are triggered once a user is authenticated.

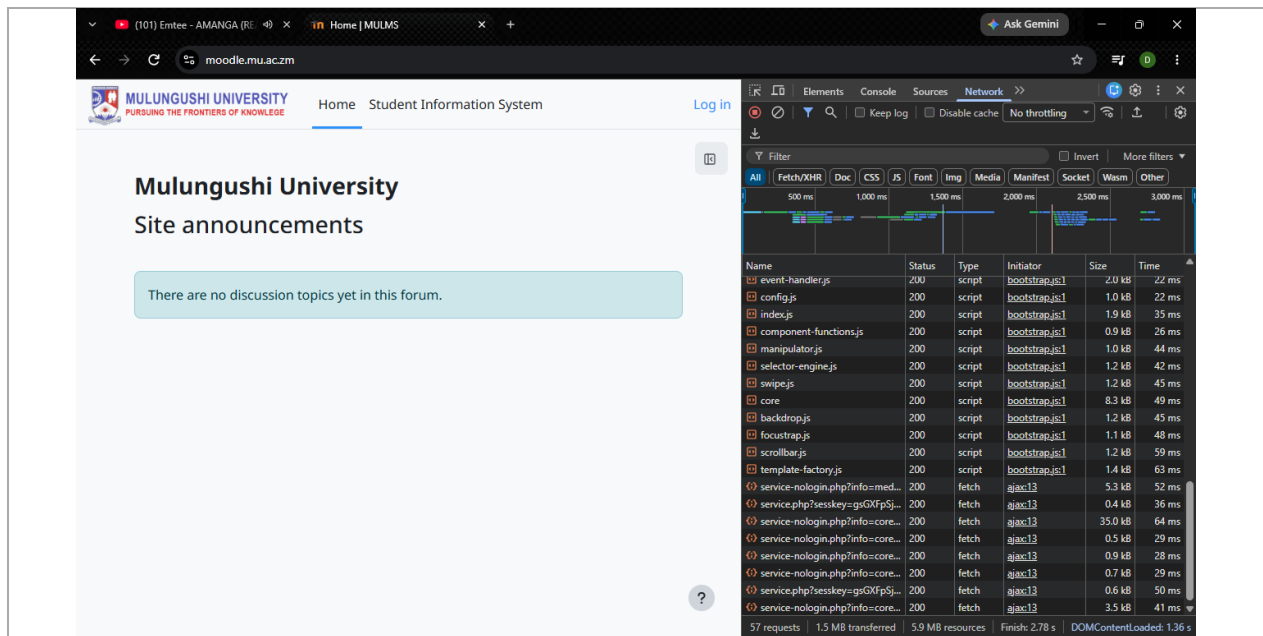


Fig. 2 — Moodle home page after login, showing the top navigation heading and a fresh set of Network requests for the authenticated view.

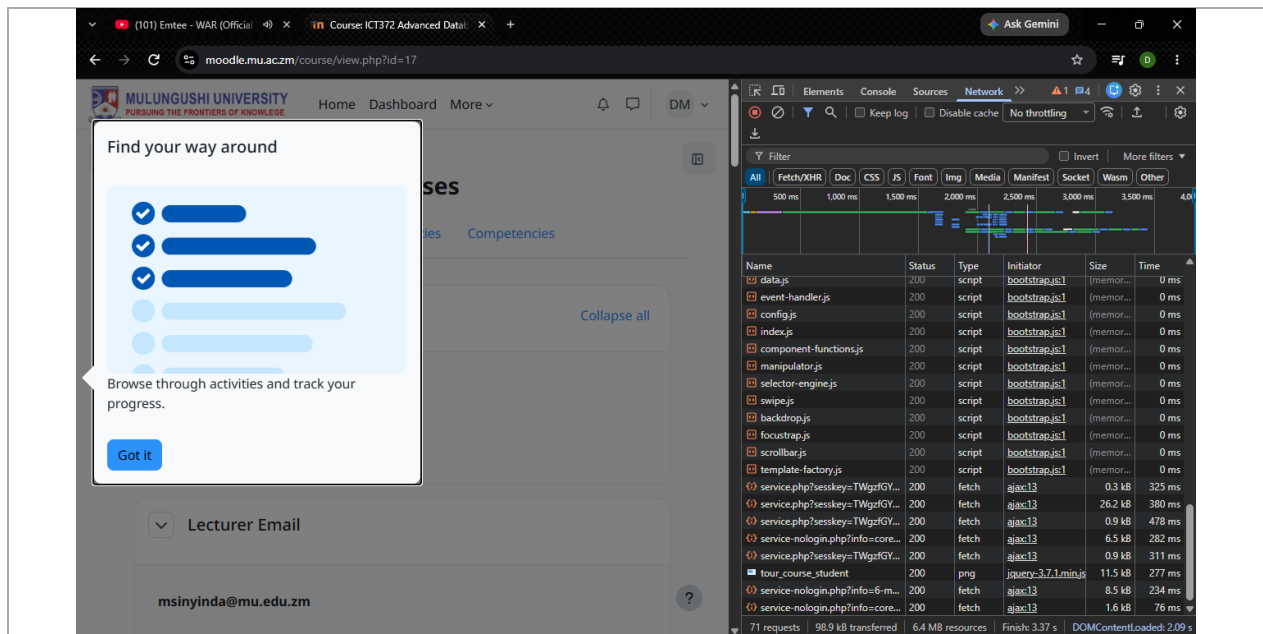


Fig. 3 — A course page showing the "Find your way around" onboarding tour and the Lecturer Email accordion control (an expandable button) alongside continued Network activity.

### 3. Page 2 — YouTube

YouTube was chosen as the second familiar page. A video page was opened, and the Elements panel was used to inspect an image element — the channel avatar — to check whether it carried alternative text.

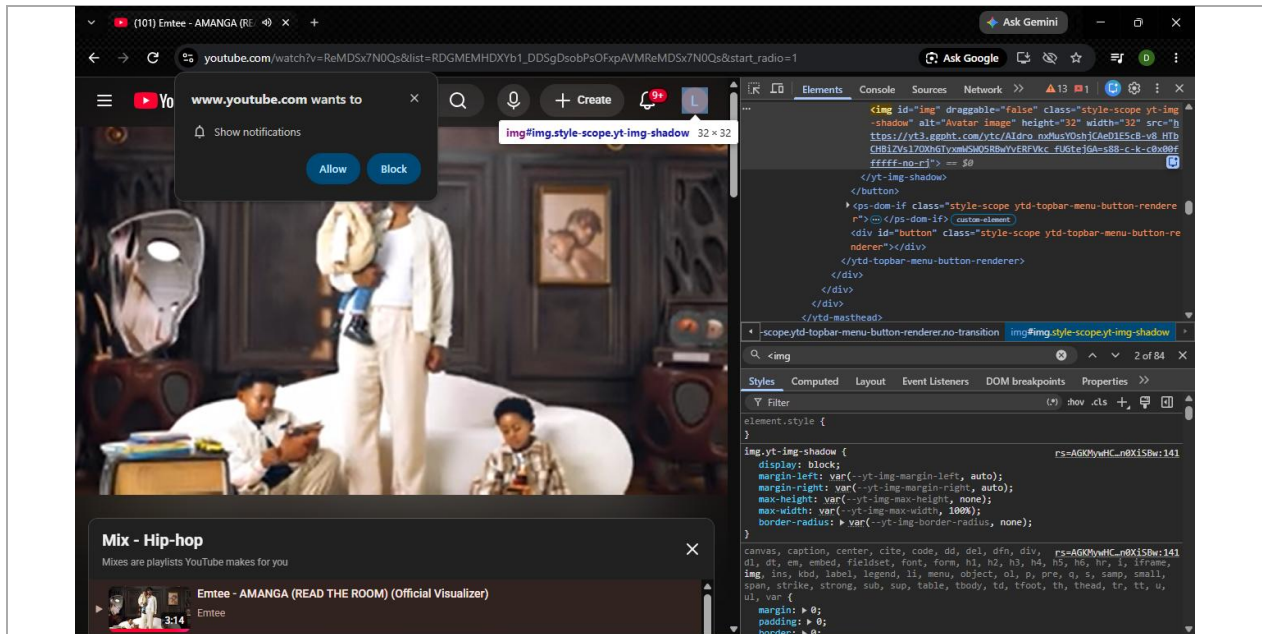


Fig. 4 — Elements panel with the avatar `<img>` element selected, showing `alt="Avatar"` already present on the element.

The Network panel was then opened and the page reloaded while the video played, to observe request volume and load timing.

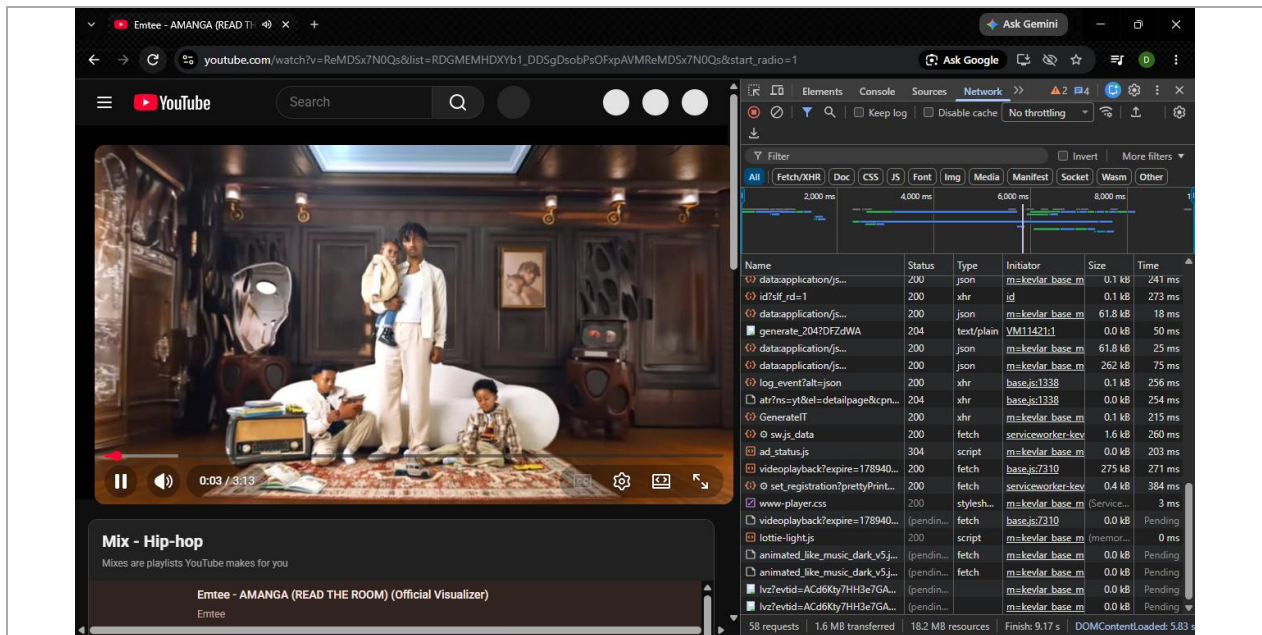


Fig. 5 — Network panel during video playback: 58 requests, 1.6 MB transferred, finish time 9.17 s.

The YouTube home feed's `<body>` element and heading structure were also inspected using the Elements panel.

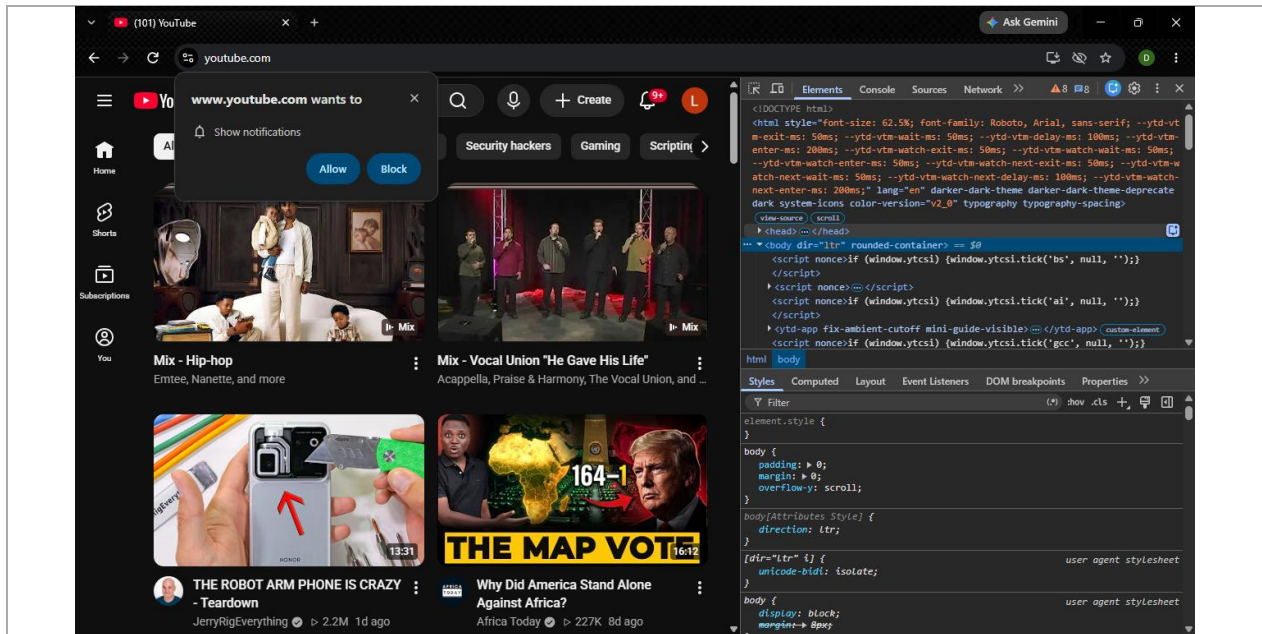


Fig. 6 — Elements panel on the YouTube home feed, inspecting the `<body>` element's computed styles beneath the video thumbnails and headings.

#### 4. Network Panel — Slowest / Failed Request

To specifically test for a slow or broken request, the same YouTube video was reopened in a mobile-emulated view and reloaded with DevTools open.

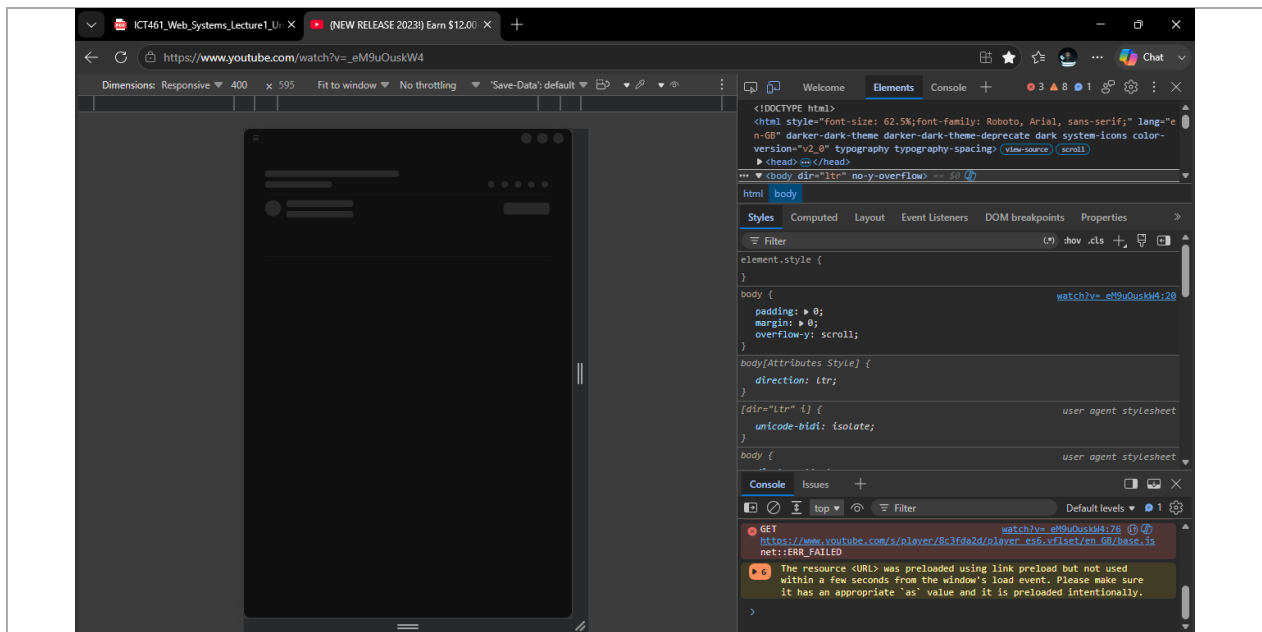


Fig. 7 — Mobile-emulated view of a YouTube video: the Console reports a failed GET request (`net::ERR_FAILED`) for the player script (`player_es6.vflset...base.js`), along with a warning that a preloaded resource was not used within a few seconds of the page load event.

This failed request was the clearest problem found: the player script did not load at all on this attempt, and a separate warning confirmed that a `<link rel="preload">` resource was fetched but never used in time, which wastes bandwidth without benefiting the page.

## 5. Diagnosis

Across both pages, the heading, button, and form-field checks passed: Moodle's Username/Password fields and Log in button were properly labelled native form controls, and the YouTube avatar image already carried an alt attribute, so no accessibility gap was found there. The Network panel told a different story. On Moodle, page loads pulled in a large number of small, separate JavaScript files (bootstrap.js, config.js, index.js, and many more) plus several fetch calls to service.php and service-nologin.php, which together explain the multi-second finish times despite each individual file being small — the bottleneck is request count rather than payload size. On YouTube, the slowest activity was the video stream itself and its many analytics/tracking calls, with finish times consistently over six seconds even on a fast connection; the mobile-emulated test additionally surfaced an outright failed request for the player's core script (net::ERR\_FAILED) alongside an unused-preload warning, which is a genuine performance defect rather than just a slow but successful load. Overall, the pages are functionally accessible at the element level, but both would benefit from combining or lazy-loading scripts to cut down request count, and YouTube's mobile experience specifically needs its preload hints and player script delivery fixed so playback does not fail to initialise.

## 6. Slowest Network Request — CNN (edition.cnn.com)

A third live page, CNN's homepage, was inspected specifically to identify the single slowest network request. The Network panel was left recording while the page continued loading its (very large) set of third-party ad and tracking resources.

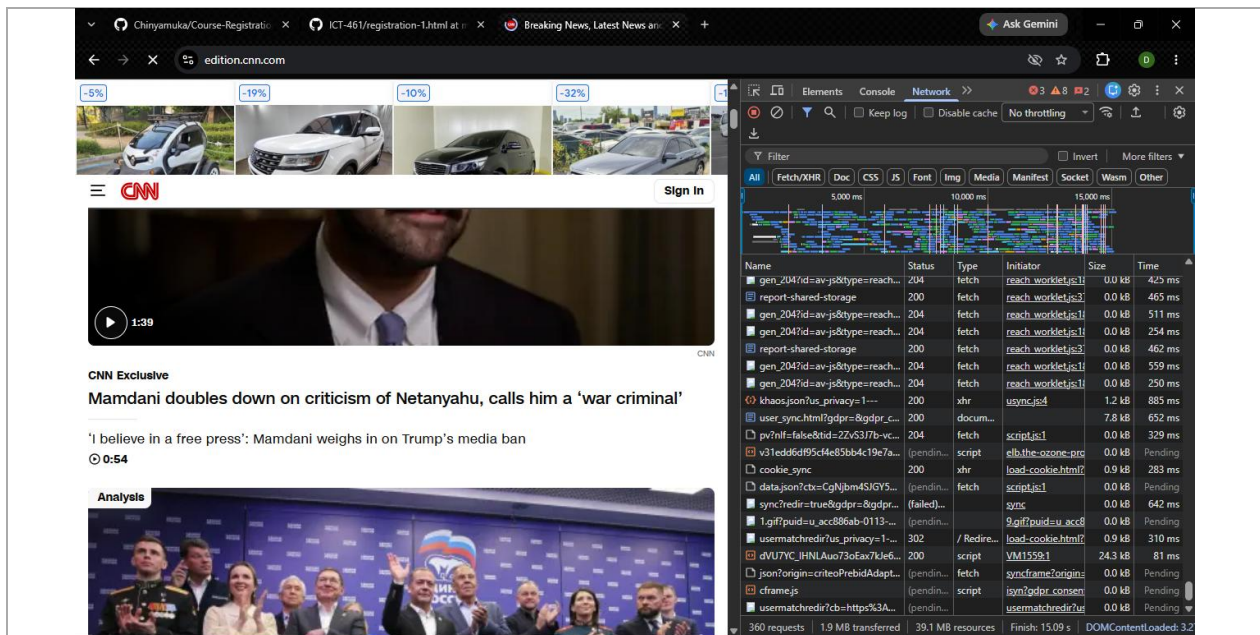


Fig. 8 — Network panel on edition.cnn.com: `khaos.json?us_privacy=1---` completed in 885 ms, the slowest finished request visible in this capture.

The slowest completed request visible was a GET to `khaos.json?us_privacy=1---`, an XHR call taking 885 ms. This endpoint belongs to a third-party ad-technology / consent-management platform: the `us_privacy` parameter is a CCPA/IAB-standard privacy-signal string, so this call is the browser reporting or checking the user's opt-out status with an external ad server before personalised ads are served. It is slow because it is a genuine network round trip to a third-party domain rather than a cached or same-origin resource — consistent with the page's wider pattern of dozens of pending ad, sync, and tracking calls layered on top of the actual article content.

Caveat: CNN's homepage keeps issuing new requests continuously (hundreds of rows, most still Pending in each capture), so this screenshot shows the slowest request visible at the moment it was taken rather than a value confirmed by sorting the full list. A stricter check would reload the page and click the Time column header to sort every request slowest-to-fastest before screenshotting the top row.